HTML – król Internetu

Laboratorium 1 | Semantyczny HTML i responsywny interfejs sklepu

Programowanie Aplikacji WWW | Wersja 2.0 | Praca indywidualna

Opracowanie Artur Samojluk | UWM w Olsztynie

Wstęp do ćwiczenia

HTML (ang. HyperText Markup Language) to hipertekstowy język znaczników służący do opisywania struktury i znaczenia treści stron WWW. Określa m.in. nagłówki, akapity, listy, hiperłącza i tabele danych oraz pozwala osadzać obrazy, filmy i dźwięk. Wygląd strony i układ elementów określa CSS. Pierwszy publiczny opis HTML ukazał się w 1991 roku, a XHTML 1.0 stał się rekomendacją W3C w 2000 roku.

Cel ćwiczenia: zrozumienie procesu tworzenia aplikacji WWW z perspektywy projektanta, od uporządkowania treści i zaprojektowania interfejsu po zapis kodu i sprawdzenie działania. Samodzielne pisanie HTML i CSS stanowi przygotowanie do zaawansowanego kodowania z użyciem technologii GPT. Pozwala świadomie formułować wymagania, oceniać wygenerowane rozwiązania i samodzielnie poprawiać błędy.

Zbuduj statyczny prototyp sklepu: katalog, stronę produktu i formularz. HTML i CSS są zakresem tego ćwiczenia; logikę aplikacji i zapis danych dobudujesz w kolejnych laboratoriach.

Zasady samodzielnej pracy

Pisz HTML i CSS ręcznie. Korzystaj z dokumentacji i narzędzi przeglądarki, ale wyłącz generowanie kodu przez GPT, Copilot i podobne narzędzia. Nie używaj gotowych szablonów, frameworków ani generatorów stron. Zwykłe podpowiedzi nazw i formatowanie w edytorze są dozwolone. Przed sprawdzeniem zmiany przewidź jej efekt; w razie błędu najpierw samodzielnie ustal przyczynę.

Przed zajęciami: przygotuj edytor, przeglądarkę, dostęp do lokalnego serwera HTTP oraz trzy niewielkie grafiki produktów. Możesz użyć własnych zdjęć. Wybierz temat sklepu, np. książki, elektronika lub sprzęt do swojego hobby.

1 Przygotuj pliki i uruchom stronę

Utwórz katalog `imie_nazwisko_nr_indeksu/lab1/`. W nim zapisz `index.html`, `produkt.html`, `kontakt.html`, `css/style.css` oraz katalog `img/` z trzema grafikami. Nazwy plików zapisuj małymi literami, bez spacji i polskich znaków.

Uruchom serwer z katalogu `lab1/`, np. poleceniem `python -m http.server 8000`, jeżeli masz Python 3. Otwórz `http://localhost:8000/`. Używaj ścieżek względnych, aby projekt działał po przeniesieniu na inny komputer.

2 Zbuduj szkielet i katalog produktów

- Każda strona: `<!doctype html>`, `html` z `lang="pl"`, `head` i `body`; w `head` kodowanie UTF-8, unikalny `title`, viewport `width=device-width, initial-scale=1` oraz odnośnik do wspólnego CSS.
- W `body` użyj `header`, `nav`, jednego `main` i `footer`. Na każdej stronie umieść jeden `h1`; kolejne nagłówki dobieraj według struktury treści. Menu ma prowadzić do wszystkich trzech stron, a bieżący link ma mieć `aria-current="page"`.
- Na `index.html` dodaj trzy karty produktów jako `article`: nazwa w nagłówku, zdjęcie, krótki opis i cena z walutą. Pierwsza karta ma prowadzić do `produkt.html`. Zdjęcia wymagają opisowego `alt` oraz atrybutów `width` i `height` zgodnych z wymiarami pliku.

Sprawdź etap: wszystkie strony i obrazy otwierają się poprawnie; katalog jest czytelny także przed dodaniem stylów.

Strona produktu formularz i style

3 Przedstaw szczegóły produktu

Na `produkt.html` opisz pierwszy produkt z katalogu. Użyj jego zdjęcia i ceny, listy co najmniej trzech cech oraz odnośnika „Zapytaj o produkt” do `kontakt.html`. Treść i cena muszą być zgodne z katalogiem.

Dodaj tabelę parametrów z co najmniej trzema wierszami danych. Zastosuj `caption`, `thead`, `tbody`, nagłówki kolumn `th` z `scope="col"` i komórki danych `td`. Tabela służy do przedstawienia danych; układ całej strony zbudujesz w CSS. W opisie użyj `strong` lub `em` tam, gdzie treść wymaga podkreślenia znaczenia.

4 Zbuduj formularz zapytania

Na `kontakt.html` przygotuj formularz demonstracyjny z `method="get"` i `action="kontakt.html"`. Używaj wyłącznie fikcyjnych danych. Dodaj następujące pola:

- e-mail: `input` typu `email`, `name="email"`, `required`;
- produkt: `select`, `name="produkt"`, `required`, trzy produkty i pierwsza opcja „Wybierz produkt” z pustym `value`;
- liczba sztuk: `input` typu `number`, `name="ilosc"`, `required`, `min="1"`, `max="5"`, `step="1"`;
- wiadomość: `textarea`, `name="wiadomosc"`, `required`, `minlength="10"`, `maxlength="500"`.

Każde pole ma mieć unikalne `id` i widoczną etykietę `label`, której `for` odpowiada temu `id`. W treści etykiet lub instrukcji podaj wymagane zakresy i oznacz pola obowiązkowe. Dodaj `button` z `type="submit"` i napisem „Sprawdź zapytanie”. Placeholder nie zastępuje etykiety.

Umieść informację: „Formularz demonstracyjny. Dane pojawią się w adresie strony; zapytanie nie zostanie zapisane ani wysłane e-mailem”. Poprawne wysłanie przeładuje stronę z parametrami w URL. Walidacja przeglądarki pomaga użytkownikowi, ale w docelowym sklepie dane trzeba też sprawdzać na serwerze.

5 Dodaj układ responsywny i obsługę klawiatury

Wszystkie reguły zapisz w `css/style.css`. Nie używaj stylów inline. Najpierw przygotuj widok na mały ekran, a następnie rozszerz go regułą `@media` dla szerokości od 768 px.

- Katalog: CSS Grid, jedna kolumna poniżej 768 px i trzy kolumny od 768 px; odstępy przez `gap`. Menu: Flexbox z zawijaniem. Nie buduj układu tabelami, spacjami ani znacznikami `br`.
- Treść: wyśrodkowany kontener o maksymalnej szerokości 1100 px, odstępy od krawędzi, `box-sizing: border-box`. Obrazy mają mieścić się w kontenerze i zachowywać proporcje. Pola formularza mają mieścić się w widoku mobilnym.
- Użyj jednej wspólnej klasy dla kart oraz co najmniej dwóch zmiennych CSS w `:root`. Zadbaj o czytelny tekst i kontrast. Zdefiniuj stan `:hover` i wyraźny `:focus-visible` dla linków i przycisku; nie usuwaj obrysu fokusu bez widocznego zamiennika.

Sprawdź etap: przy 375 px oraz 1280 px treść nie jest ucięta, a cała strona nie wymaga przewijania w poziomie.

Testy i zaliczenie

6 Sprawdź działanie i wyjaśnij wynik

Wykonaj próby. W `TESTY.md` zapisz wersję przeglądarki, oczekiwany i zaobserwowany wynik każdej próby oraz ewentualną poprawkę.

- **Struktura:** sprawdź trzy pliki HTML w Nu HTML Checker. Usuń błędy; przeczytaj i wyjaśnij pozostałe ostrzeżenia. Wyłącz CSS w DevTools i sprawdź kolejność nagłówków oraz treści.
- **Widoki:** w trybie responsywnym sprawdź 375 px i 1280 px. Zmień szerokość przez granicę 768 px. Potwierdź liczbę kolumn, brak ucięcia formularza i proporcje obrazów.
- **Klawiatura:** przejdź przez menu i formularz za pomocą Tab oraz Shift+Tab, aktywuj link Enterem. Sprawdź kolejność i widoczność fokusu. Kliknięcie etykiety ma aktywować właściwe pole.
- **Walidacja:** kolejno spróbuj wysłać pusty formularz, błędny e-mail, liczbę 0 i 6 oraz ręcznie wpisaną wiadomość krótszą niż 10 znaków. Pozostałe pola w danej próbie wypełnij poprawnie. Każda próba ma zostać zatrzymana. Następnie wyślij poprawny zestaw i wskaż cztery parametry w URL.
- **Eksperyment:** przewidź efekt usunięcia `name` z pola e-mail, pozostawiając `id` i `label`. Usuń atrybut, wyślij poprawny formularz i porównaj URL. Zapisz wyjaśnienie, potem przywróć `name`. W DevTools pokaż żądanie formularza i jego metodę GET.

Oddanie i sprawdzenie samodzielności

Do rozpoczęcia kolejnych zajęć umieść katalog `lab1/` z ukończonym laboratorium w swoim publicznym repozytorium GitHub. Przekaż prowadzącemu link do tego katalogu. Repozytorium jest źródłem prawdy o projekcie i terminowości pracy. Terminowość jest weryfikowana na podstawie wersji dostępnej w repozytorium przed rozpoczęciem kolejnych zajęć. Historia commitów dokumentuje postęp pracy.

Zamieść cały kod, grafiki, `TESTY.md` oraz `README.md` z instrukcją uruchomienia i źródłami grafik. Sprawdź uruchomienie projektu po pobraniu repozytorium.

Przy oddaniu wykonaj samodzielnie zmianę wskazaną przez prowadzącego, np. zmianę progu responsywności, dodanie czwartej karty lub zmianę limitu ilości. Przed edycją wskaż odpowiedni kod i przewidź efekt; po edycji sprawdź działanie. Wyjaśnij różnicę między `id` i `name` oraz przeznaczenie tabeli danych.

Dokumentacja do pracy

HTML Standard | CSS Grid | Etykiety formularzy

Walidacja formularzy | Nu HTML Checker